

Exercice 2-2 — Une unité systemd pour le script d'inventaire

Module : 03 — CL-LINUX Linux pour l'admin cloud Durée estimée : 45 min Difficulté : 3 / 5 Type : Exercice d'application

Objectifs pédagogiques

À la fin de cet exercice, vous serez capable de :

- Installer un script dans `/usr/local/bin` et le rendre exécutable
- Écrire de zéro une unité systemd `Type=oneshot` exécutée à chaque démarrage
- Dérouler le cycle complet : `daemon-reload` → `start` → `status` → `enable`
- Vérifier après un reboot qu'une unité s'est bien exécutée (`journalctl -u`)

Prérequis

- Avoir suivi la partie « Écrire une unité » du module 03 (jour 2)
- Bloc 02 CL-PYTHON (le script fourni est du Python standard, vous devez pouvoir le relire)
- Environnement : VM `srv-linux` (Ubuntu Server 24.04 LTS sous Multipass) — connectez-vous avec `multipass shell srv-linux` depuis le poste hôte
- Outils : `nano`, `python3` (préinstallé sur Ubuntu 24.04), `systemctl`, `journalctl`

Contexte

Au bloc CL-PYTHON, vous aviez écrit un script d'inventaire système. L'équipe d'exploitation veut maintenant l'industrialiser : **à chaque démarrage du serveur**, le script doit produire un état des lieux (nom de machine, uptime, espace disque) dans `/var/lib/inventaire/rapport.txt`. Quand un serveur redémarre à 3 h du matin dans le cloud, personne n'est devant l'écran : c'est systemd qui doit lancer le script, pas un humain. Sur vos futures instances EC2, ce motif « script au boot piloté par une unité » est un grand classique.

Voici le script, validé par l'équipe (stdlib uniquement, rien à installer) :

```
#!/usr/bin/env python3
"""Inventaire système : hostname, uptime, disque ->
/var/lib/inventaire/rapport.txt"""

import shutil
import socket
from datetime import datetime

RAPPORT = "/var/lib/inventaire/rapport.txt"

def lire_uptime():
    with open("/proc/uptime") as f:
        secondes = float(f.read().split()[0])
```

```
heures, reste = divmod(int(secondes), 3600)
minutes = reste // 60
return f"{heures} h {minutes:02d} min"

def main():
    total, utilise, libre = shutil.disk_usage("/")
    go = 1024 ** 3
    lignes = [
        f"Rapport genere   : {datetime.now():%Y-%m-%d %H:%M:%S}",
        f"Machine           : {socket.gethostname()}",
        f"Uptime              : {lire_uptime()}",
        f"Disque / total       : {total / go:.1f} Go",
        f"Disque / occupe      : {utilise / go:.1f} Go ({utilise / total * 100:.0f} %)",
        f"Disque / libre       : {libre / go:.1f} Go",
    ]
    with open(RAPPORT, "w") as f:
        f.write("\n".join(lignes) + "\n")
    print(f"Rapport ecrit dans {RAPPORT}")

if __name__ == "__main__":
    main()
```

Énoncé

Toutes les commandes s'exécutent dans la VM (`multipass shell srv-linux`).

Partie 1 — Installer et tester le script à la main

Règle d'or : **jamais d'unité systemd autour d'un script qu'on n'a pas d'abord vu fonctionner à la main.**

1. Installez le script ci-dessus dans `/usr/local/bin/inventaire.py` (avec `sudo nano`, ou en le collant via `sudo tee`).
2. Rendez-le exécutable pour tout le monde, lisible mais modifiable uniquement par root.
3. Créez le répertoire de destination `/var/lib/inventaire`.
4. Exécutez le script à la main (il écrit dans `/var/lib`, il lui faut donc les droits de root), puis affichez le rapport produit.

Résultat attendu :

```
sudo /usr/local/bin/inventaire.py
cat /var/lib/inventaire/rapport.txt
```

```
Rapport ecrit dans /var/lib/inventaire/rapport.txt
Rapport genere   : 2026-07-07 14:20:33
Machine          : srv-linux
```

```
Uptime           : 2 h 05 min
Disque / total   : 19.3 Go
Disque / occupe  : 2.1 Go (11 %)
Disque / libre   : 17.2 Go
```

(vos valeurs différeront, la forme doit être celle-ci).

Partie 2 — Écrire l'unité

1. Créez le fichier `/etc/systemd/system/inventaire.service` avec :
 - une section `[Unit]` : une **Description** claire en français ;
 - une section `[Service]` : `Type=oneshot` (le script fait son travail puis se termine — ce n'est pas un démon qui tourne en continu) et le **ExecStart** qui lance le script **via l'interpréteur en chemin absolu** (`/usr/bin/python3 /usr/local/bin/inventaire.py`) ;
 - une section `[Install]` : `WantedBy=multi-user.target` pour un lancement à chaque démarrage.
2. Faites connaître la nouvelle unité à systemd.
3. Supprimez le rapport existant (`sudo rm /var/lib/inventaire/rapport.txt`) pour prouver que c'est bien l'unité qui va le recréer, puis démarrez-la.
4. Contrôlez son état et ses journaux. **Question** : pourquoi `systemctl status` n'affiche-t-il pas **active (running)** comme pour nginx ?

Résultat attendu :

```
sudo systemctl start inventaire.service
systemctl status inventaire.service --no-pager
```

affiche **Deactivated successfully** / état **inactive (dead)** avec (`code=exited, status=0/SUCCESS`) dans la ligne **Process:**, et `cat /var/lib/inventaire/rapport.txt` montre un rapport tout frais.

Partie 3 — Au boot, sans intervention humaine

1. Activez l'unité au démarrage et vérifiez qu'elle est bien **enabled**.
2. Notez l'heure courante (**date**), puis redémarrez la VM : `sudo reboot`. Attendez quelques secondes et reconnectez-vous (`multipass shell srv-linux`).
3. Prouvez que le script a tourné **pendant le boot, sans vous** :
 - le rapport est daté d'après le reboot et l'uptime affiché est de quelques minutes au plus ;
 - les journaux de l'unité sur ce boot en témoignent (`journalctl -u inventaire -b --no-pager`).

Résultat attendu : `journalctl -u inventaire -b --no-pager` contient **Rapport écrit dans /var/lib/inventaire/rapport.txt** horodaté dans les secondes qui suivent le démarrage, et le rapport affiche un uptime de l'ordre de **0 h 00 min**.

Indices (à consulter si bloqué)

- Indice 1 — l'unité minimale qui fonctionne

- Indice 2 — systemd ne voit pas votre unité ou vos modifications
- Indice 3 — status affiche inactive (dead), est-ce grave ?

Pour aller plus loin (bonus)

1. **RemainAfterExit** : ajoutez `RemainAfterExit=yes` dans `[Service]`, rechargez, redémarrez l'unité et comparez la sortie de `systemctl status inventaire` avec celle d'avant. À quoi cette directive peut-elle servir quand d'autres unités dépendent de celle-ci ?
2. **Sentinelle disque** : créez une seconde unité `verif-disque.service` (Type=oneshot, lancée au boot elle aussi) qui **échoue** si le disque `/` est rempli à plus de 90 %. Le scripting shell arrive au jour 3, voici donc le script à installer dans `/usr/local/bin/verif-disque.sh` :

```
#!/usr/bin/env bash
SEUIL=90
USAGE=$(df --output=pcent / | tail -n 1 | tr -d ' %')
if [ "$USAGE" -gt "$SEUIL" ]; then
    echo "ALERTE : disque / rempli a ${USAGE}% (seuil ${SEUIL}%)"
    exit 1
fi
echo "OK : disque / rempli a ${USAGE}% (seuil ${SEUIL}%)"
```

Vérifiez qu'elle réussit (le disque de la VM est loin de 90 %), puis abaissez temporairement `SEUIL` à 5 pour la voir échouer : que montrent alors `systemctl status verif-disque` et `systemctl --failed` ? Remettez `SEUIL=90` à la fin.